

Funciones y testeos unitarios

1. Declaración de funciones	2
2. Ventajas del uso de funciones	2
3. Código reutilizable	3
4. Modelo de caja negra y testeo unitarios	6
4.1. Testeo simple (volcado de la salida a la consola)	7
4.2. Testeo automático (usando valores de entrada y esperados)	8
4.3. Testeo mediante el uso de librerías	10

1. Declaración de funciones

Para declarar una función en JavaScript utilizaremos la palabra reservada **function**:

```
function <nombre de función>(parametro1, parametro2, ..., parametro n)
{
    <código de la función>
}
```

Los parámetros son los datos de entrada que la función necesita. Dentro de la función escribiremos el código necesario para que realice los cálculos oportunos. El valor de salida de la función lo retornaremos haciendo uso de la palabra reservada **return**.

Por ejemplo, una función a la que le pasemos tres notas y nos devuelva la nota promedio podría tener el siguiente código:

```
function notaPromedio(nota1, nota2, nota3){
    let suma = nota1 + nota2 + nota3;
    let promedio = suma / 3;
    return promedio;
}
```

2. Ventajas del uso de funciones

Las funciones nos proporcionan múltiples ventajas, entre ellas:

- Permiten escribir código reutilizable, que será utilizado en distintas partes de nuestra aplicación, así como también en otros proyectos
- Permite dividir una tarea compleja en otras pequeñas que serán más fáciles de abordar (divide y vencerás)
- El código es más fácil de entender y mantener, ya que cada función se encargará de una tarea específica. También nos facilitarán las mejoras y ampliaciones.
- Permiten hacer testeos robustos, usando las pruebas de caja negra (proporcionando valores de entrada a la función y los valores esperados de retorno). También se llaman pruebas unitarias.

3. Código reutilizable

Como hemos mencionado anteriormente una de las grandes ventajas de las funciones es que nos permiten crear código reutilizable, que podremos llamar desde distintos puntos de nuestro programa.

Veamos un ejemplo de un programa de JavaScript que pida a un usuario el radio y la altura de un cilindro y calcule el volumen del mismo.

El volumen de un cilindro viene dado por la fórmula:

$$V = \pi \cdot r^2 \cdot h$$

Donde V es el volumen, r el radio y h la altura

Podríamos escribir un programa en JavaScript similar al siguiente:

```
let radio = parseFloat (prompt("Introduce el radio del cilindro"));
let altura = parseFloat (prompt("Introduce la altura del cilindro"));
let volumen = Math.PI * Math.pow(radio,2) * altura;
alert(volumen);
```

Una vez creado nuestro programa, como testeo previo, podríamos alguna página web que calcule el volumen de un cilindro, por ejemplo:

https://es.onlinemschool.com/math/assistance/figures_volume/cylinder/

El programa funciona correctamente, pero no hemos utilizado código reutilizable, por lo que si posteriormente tenemos que volver a hacer un cálculo similar tendremos que volver a repetir el cálculo del volumen. Este cálculo es muy simple y podríamos pensar que no importa volver a escribir dicho cálculo cada vez que lo necesitemos, pero si por ejemplo hemos cometido un error en la ecuación tendremos que buscar todas las partes del código y corregirlas.

Sin embargo, si queremos calcular el volumen de dos cilindros usando una función nuestro código podría quedar de la siguiente manera:

```
function volumenCilindro(){
  let radio = parseFloat (prompt("Introduce el radio del cilindro"));
  let altura = parseFloat (prompt("Introduce la altura del cilindro"));
  let volumen = Math.PI * Math.pow(radio,2) * altura;
  alert(volumen);
}

//Llamamos una vez a nuestra función
volumenCilindro();

//En otra parte de nuestro código podemos volver a llamar a nuestra función
volumenCilindro();
```

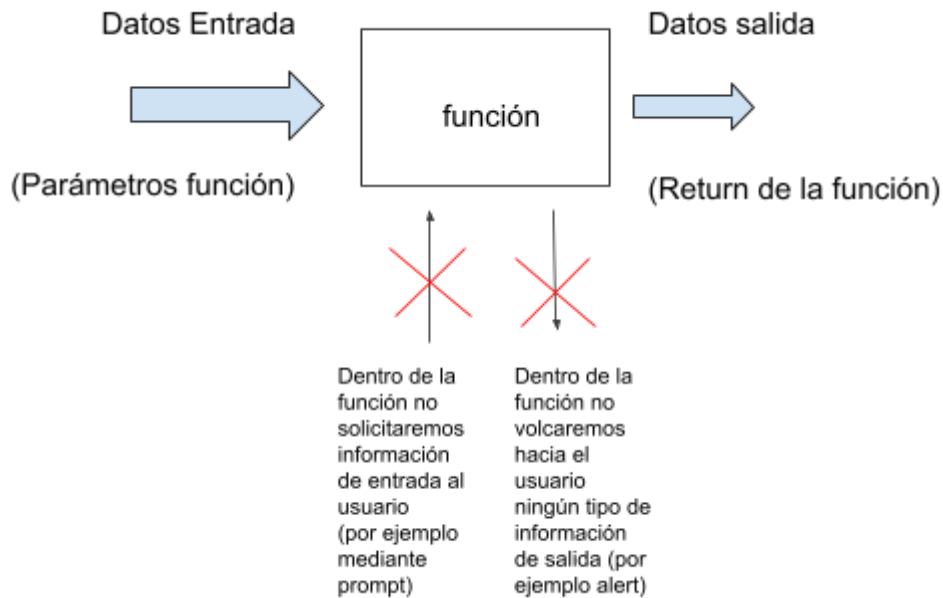
Como puedes comprobar en el ejemplo anterior hemos encapsulado en una función el código específico para calcular el volumen. Pero **este código es muy rígido**, y por tanto es reutilizable pero en menor grado, ya que los datos para el cálculo siempre se solicitan a partir de un prompt y el volumen siempre se muestra al usuario con un alert.

Debemos escribir código que verdaderamente sea reutilizable y nos de igual el origen de los datos (que los proporcione el usuario, que los obtengamos de una base de datos, etc.). Y los datos de salida de la función también debemos pensar que no siempre vamos a querer mostrarlos con un alert (podríamos querer volcarlos a un archivo, escribirlos en una base de datos, enviarlos a otro servidor, etc.).

Para escribir **código reutilizable** vamos a seguir los siguientes **principios de diseño** de la función:

- Todos los datos que necesite nuestra función para el cálculo se lo pasaremos en forma de argumentos
- Los datos de salida de la función siempre serán en forma de return
- Desde el interior de la función no escribiremos ninguna instrucción que obtenga datos de entrada adicionales (por ejemplo no usaremos un prompt dentro de la función)
- Desde el interior de la función nos limitaremos a devolver los datos de salida en el return (por ejemplo nunca usaremos un alert dentro de la función).

Los principios anteriores los podemos ver visualmente reflejados en el siguiente gráfico:



El código reutilizable de nuestra programa para calcular el volumen de dos cilindros usando código reutilizable quedaría similar a:

```
function volumenCilindro(radio, altura){  
    let volumen = Math.PI * Math.pow(radio,2) * altura;  
    return volumen;  
}  
  
//Cilindro 1  
let r1 = parseFloat (prompt("Introduce el radio del cilindro1"));  
let a1 = parseFloat (prompt("Introduce la altura del cilindro1"));  
let v1 = volumenCilindro(r1, a1);  
alert("Volumen del cilindro1 es " + v1);  
  
//Cilindro 2  
let r2 = parseFloat (prompt("Introduce el radio del cilindro2"));  
let a2 = parseFloat (prompt("Introduce la altura del cilindro2"));  
let v2 = volumenCilindro(r2, a2);  
alert("Volumen del cilindro1 es " + v2);
```

4. Modelo de caja negra y testeos unitarios

Dado un sistema que procesa una información de entrada y nos proporciona una salida, el modelo de caja negra consiste en despreocuparnos del interior del sistema (cómo procesa la información) y centrarnos solo en la entrada y la salida del mismo.



Para ver si el sistema funciona correctamente podemos inyectarle ciertas entradas y comprobar si la salida es la esperada.

En el mundo de la programación si seguimos los principios que hemos visto en apartado anterior referente a crear código reutilizable usando funciones podremos ver dicha función como una caja negra y someterla a test de entradas y las correspondientes salidas esperadas. A este tipo de tests se le denomina de **testeos unitarios** (ya que analizamos un bloque de código particular del programa, no el funcionamiento global del programa en que interactuarán varios bloques).

¿Cómo podríamos preparar un test de pruebas unitarias para nuestra función anterior **volumenCono**? Pues muy sencillo. Tenemos que preparar una serie de datos de entrada (en nuestro caso de radios y alturas de cilindros y sus correspondientes volúmenes. Para obtener dichos datos podríamos usar distintos métodos:

- Una calculadora científica que calcule el volumen de un cilindro
- Otra aplicación informática que haga dicho cálculo
- Una web online (eso sí, no vale cualquiera, debe proporcionarnos cierta confianza, ya que si usamos la primera que encontremos podría haber sido desarrollada por alguien que no la haya testeado y no funcione correctamente).

Nosotros vamos a usar una web online de OnlineMSchool, en la cual confiamos que los cálculos son los correctos:

https://es.onlinemschool.com/math/assistance/figures_volume/cylinder/

Preparamos una tabla con los valores que queramos de entrada (distintos radios y alturas) y usamos la web online para calcular el volumen:

VALORES DE ENTRADA		VALOR ESPERADO (Redondeados a tres decimales)
radio	altura	volumen
4.5	6.7	426.236
6.1	4.3	502.664
17.8	9	8958.440

En nuestras pruebas unitarias las dos primeras columnas (radio y altura) serán los datos de entrada y la tercera columna (el volumen) será el valor esperado (el que deberías devolver la función si realmente está realizando el cálculo correctamente).

Una vez que tenemos preparados los datos de la tabla anterior podríamos preparar un programa que se encargue de llamar a la función a testear con los valores de entrada y compararlos con los esperados.

4.1. Testeo simple (volcado de la salida a la consola)

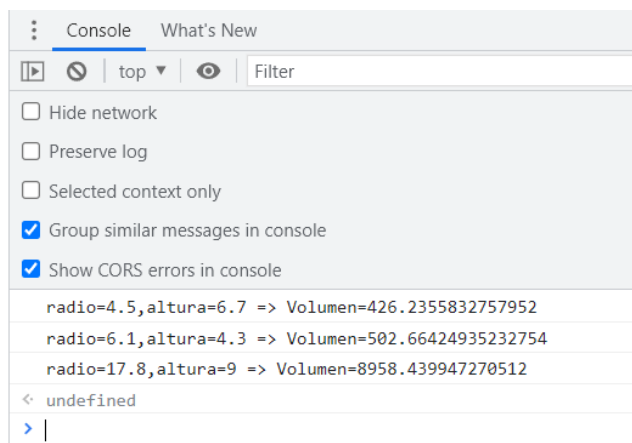
Podríamos implementar un código de testeo simple que se limite a encapsular en algún tipo de estructura (por ejemplo un array) los datos de entrada a la función y posteriormente volcar la salida a pantalla. De manera manual comprobaríamos la tercera columna de la tabla con las salida para ver si nuestra función funciona bien.

```
function volumenCilindro(radio, altura){  
    let volumen = Math.PI * Math.pow(radio,2) * altura;  
    return volumen;  
}  
  
// Encapsulamos los valores de entrada en un array de objetos literales  
let valores = [{radio:4.5, altura:6.7}, {radio: 6.1, altura: 4.3}, {radio:  
17.8, altura: 9}];  
let salida;
```

```
// Con el bucle llamamos a nuestra función con cada uno de los pares radio,
altura del array

for (let i=0; i < valores.length; i++){
    salida = volumenCilindro(valores[i].radio, valores[i].altura);
    //Volcamos a la consola la salida de la función
    console.log("radio="+valores[i].radio + ",altura="+valores[i].altura + "
=> Volumen=" + salida);
}
```

Al ejecutar el programa obtendremos la siguiente salida en la consola del navegador:



Comparando los valores esperados de la tabla con los obtenidos en Volumen en la consola vemos que coincide.

4.2. Testeo automático (usando valores de entrada y esperados)

Ahora vamos a preparar una versión mejorada para testear nuestra función de manera que usaremos tanto los valores de entrada como la salida esperada de la función. Los datos esperados de la tabla están redondeados a tres decimales, por lo que tendremos que tenerlo en cuenta en nuestro código para el testeo:

```
function volumenCilindro(radio, altura){
    let volumen = Math.PI * Math.pow(radio,2) * altura;
    return volumen;
}

// Función auxiliar para redondear un número
function redondearDecimales(numero,decimales){
    return Math.round(numero * Math.pow(10,decimales)) /
    Math.pow(10,decimales);
}
```



```
// Encapsulamos los valores de entrada y la salida esperada en un array de
objetos literales

let valores = [{radio:4.5, altura:6.7, volumen:426.236}, {radio: 6.1,
altura: 4.3, volumen: 502.664}, {radio: 17.8, altura: 9, volumen: 8958.440}];

let salida;

// Con el bucle llamamos a nuestra función con cada uno de los pares radio,
altura del array

for (let i=0; i < valores.length; i++){

    salida = redondearDecimales(volumenCilindro(valores[i].radio,
valores[i].altura), 3);

    // Comparamos el valor esperado con el devuelto por la función

    if(salida == valores[i].volumen)

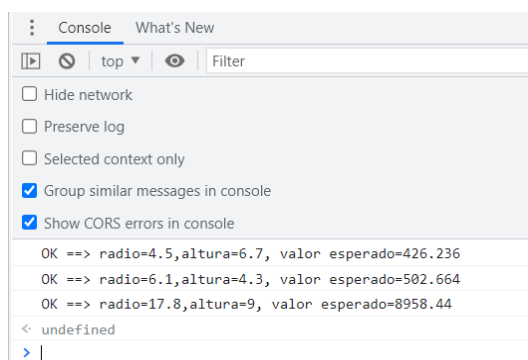
        console.log("OK ==> " + "radio="+valores[i].radio +
",altura="+valores[i].altura + ", valor esperado=" + valores[i].volumen);

    else

        console.log("FALLA ==> " + "radio="+valores[i].radio +
",altura="+valores[i].altura + ", valor esperado=" + valores[i].volumen);

}
```

En la consola obtenemos la siguiente salida:



4.3. Testeo mediante el uso de librerías

Otra posibilidad para hacer testeos de pruebas unitarias es utilizar librerías como [Jasmine](#). En la siguiente sección abordaremos con profundidad el testeo con dicha herramienta.